

Worksheet 3 — Cryptography Used Correctly (and Misused) (3 hrs)

Course: Software Security (KOSEN69) · **Week 3 Aligned to:** OWASP 2025 A04 Cryptographic Failures · CWE-327, CWE-916, CWE-330, CWE-798 **Signature game:** "Capture the Hash" (recover plaintext from weak hashes)

Ethics note: Crack only the hashes provided in `hashes.txt` on your own machine. Password-cracking against accounts or systems you don't own is illegal. Wordlists and recovered values stay inside the lab VM.

Part 1 — Student Information

Name	Student ID	Date	Group	AI Tool
Thura Aung	6631503094	16.8.2026	-	ChatGPT

Part 2 — Lecture Questions

Answer in your own words (2–4 sentences each).

1. Distinguish hashing, encryption, and encoding — and give one job each is the wrong tool for.
 - 1.Hashing, encryption, and encoding:Hashing turns data into a fixed value and is mainly used for checking integrity or storing passwords securely. Encryption hides data using a key and can be reversed, while encoding only changes the format of data. Hashing is wrong for data you need to recover, encryption is wrong for password storage, and encoding is wrong for protecting sensitive information.
2. Why is a fast hash like MD5/SHA-1 a bad choice for storing passwords, and what should be used instead?
 - 2.MD5/SHA-1 for passwords:MD5 and SHA-1 are very fast, so attackers can try millions of password guesses quickly. Passwords should use slow password-hashing algorithms such as Argon2id, bcrypt, or scrypt.
3. What is a salt, what attack does it defeat, and why must it be unique per password?
 - 3.Salt:A salt is a random value added to a password before hashing. It prevents attackers from easily using precomputed tables such as rainbow tables. Each password needs a unique salt so identical passwords do not produce identical hashes.
4. Why does AES-ECB leak structure, and what does an authenticated mode like AES-GCM add?
 - 4.AES-ECB vs AES-GCM: AES-ECB encrypts identical blocks into identical ciphertext blocks, so patterns in the original data can still be visible. AES-GCM hides these patterns and also checks that the encrypted data has not been changed.
5. What's the difference between `random` and a CSPRNG (e.g. `secrets`), and where does it matter?

- 5.random vs CSPRNG:random is designed for simulations and normal applications, not for security, because its output may be predictable. A CSPRNG such as Python's secrets produces much harder-to-predict values and should be used for passwords, reset tokens, session IDs, and security keys.

“We used AES” says nothing.

OWASP A04 · four independent decisions, four separate fixes.



One cipher name answers none of these four questions.

AES-GCM under a hardcoded key is still CWE-798.

Part 3 — Hands-on Lab (180 min)

Learning goals: exploit four crypto misuses, then remediate them with a vetted KDF, authenticated encryption, and a CSPRNG. **Prerequisites:** Docker (or local Python 3.12); `hashcat` or `john`; the `rockyou.txt` wordlist.

Environment setup

```
cd labs/week03-cryptography
docker compose up          # installs pycryptodome + argon2-cffi, runs both
scripts
# or locally:
pip install pycryptodome argon2-cffi
python vulnerable_crypto.py # see the md5 hash, repeated ECB blocks, 6-digit token
```

Targets: `vulnerable_crypto.py` (the misuses), `hashes.txt` (four unsalted MD5s), and `solution_skeleton.py` (the fix).

What to submit per task: the command/payload run + a screenshot of the result + a 2–3 sentence mitigation.

Task 0 — Onboarding (5 min) · *Goal:* see the misuse output. *Steps:* run `python vulnerable_crypto.py`; note the md5 digest, the identical ECB ciphertext blocks, and the short token. *Deliverable:* screenshot of the program output.

(Task 0)

```
Thura Aung | 6631503094 | 2026-08-16 14:41:29 +07:00
PS C:\software-security\labs\week03-cryptography> python .\vulnerable_crypto.py
md5: 482c811da5d5b4bc6d497ffa98491e38
ecb: 3bfd04cc0d7ed55358e2cbe19de213833bfd04cc0d7ed55358e2cbe19de21383377222e061a924c591cd9c27ea163ed4
token: 436336
PS C:\software-security\labs\week03-cryptography> |
```

The vulnerable program ran successfully and showed MD5 hashing, AES-ECB encryption, and a 6-digit random token. This confirms the insecure crypto behaviors that will be tested and fixed in the next tasks.

Task 1 — Capture the Hash (30 min) · *Goal:* recover the passwords. *Steps:* strip the comment lines from `hashes.txt`, then run `hashcat -m 0 hashes.txt rockyou.txt` (or the `john --format=raw-md5` equivalent); recover all four plaintexts. *Deliverable:* screenshot of the cracked results (mask any real-looking value). Note in one line why unsalted MD5 fell so fast (CWE-916/327).

aes-modes

(Task 1)

```

ATTENTION! Pure (unoptimized) backend kernels selected.
Pure kernels can crack longer passwords, but drastically reduce performance.
If you want to switch to optimized kernels, append -O to your commandline.
See the above message to find out about the exact limits.

Watchdog: Temperature abort trigger set to 90c

Host memory allocated for this attack: 880 MB (2929 MB free)

Dictionary cache hit:
* Filename..: C:\software-security\labs\week03-cryptography\rockyou.txt
* Passwords.: 14344384
* Bytes.....: 139921497
* Keyspace..: 14344384

482c811da5d5b4bc6d497ffa98491e38:password123
e10adc3949ba59abbe56e057f20f883e:123456
25f9e794323b453885f5181f1b624d0b:123456789
5f4dcc3b5aa765d61d8327deb882cf99:password

Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 0 (MD5)
Hash.Target....: C:\software-security\labs\week03-cryptography\hashes_clean.txt
Time.Started...: Tue Aug 18 21:58:47 2026 (1 sec)
Time.Estimated.: Tue Aug 18 21:58:48 2026 (0 secs)
Kernel.Feature..: Pure Kernel (password length 0-256 bytes)
Guess.Base.....: File (C:\software-security\labs\week03-cryptography\rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#01.....: 0 H/s (4.40ms) @ Accel:1024 Loops:1 Thr:64 Vec:1
Speed.#02.....: 12302.6 kH/s (6.56ms) @ Accel:1024 Loops:1 Thr:64 Vec:1
Speed.#*.....: 12302.6 kH/s
Recovered.....: 4/4 (100.00%) Digests (total), 4/4 (100.00%) Digests (new)
Progress.....: 458752/14344384 (3.20%)
Rejected.....: 0/458752 (0.00%)
Restore.Point...: 0/14344384 (0.00%)
Restore.Sub.#01.: Salt:0 Amplifier:0-1 Iteration:0-1
Restore.Sub.#02.: Salt:0 Amplifier:0-1 Iteration:0-1
Candidate.Engine.: Device Generator
Candidates.#01...: tunesien -> princess-xx
Candidates.#02...: 123456 -> tuneup
Hardware.Mon.#01.: Temp: 40c Util: 5% Core: 947MHz Mem:3868MHz Bus:8
Hardware.Mon.#02.: Util: 8% Core: 400MHz Mem:1600MHz Bus:16

Started: Tue Aug 18 21:58:46 2026
Stopped: Tue Aug 18 21:58:48 2026
PS C:\Tools\hashcat-7.1.2\hashcat-7.1.2> .\hashcat.exe -m 0 C:\software-security\labs\week03-cryptography\hashes_clean.txt --show
482c811da5d5b4bc6d497ffa98491e38:password123
e10adc3949ba59abbe56e057f20f883e:123456
25f9e794323b453885f5181f1b624d0b:123456789
5f4dcc3b5aa765d61d8327deb882cf99:password
PS C:\Tools\hashcat-7.1.2\hashcat-7.1.2> Write-Output "Thura Aung | 6631503094 | $(Get-Date -Format 'yyyy-MM-dd HH:mm:ss K')"
Thura Aung | 6631503094 | 2026-08-18 22:00:01 +07:00
PS C:\Tools\hashcat-7.1.2\hashcat-7.1.2> |

```

Unsalted MD5 was cracked very quickly because MD5 is designed to be fast and the hashes did not use unique salts, allowing Hashcat to compare millions of common password guesses efficiently (CWE-916/327).

Mitigation:

Passwords should be stored using Argon2id instead of MD5. Argon2id uses salts and is intentionally slower, making password cracking much harder.

Task 2 — ECB structure leak (20 min) · Goal: prove ECB leaks. Steps: call `encrypt_ecb(b"A"*16 + b"A"*16)` from `vulnerable_crypto.py` and show the two 16-byte ciphertext blocks are identical; explain how this leaks plaintext structure (CWE-327). Deliverable: hex output highlighting the repeated block.

(Task 2)

```
PS C:\software-security\labs\week03-cryptography> Write-Host "Thura Aung | 6631503094 | $(Get-Date -Format 'yyyy-MM-dd HH:mm:ss zzz')"
```

```
Thura Aung | 6631503094 | 2026-08-16 15:51:15 +07:00
```

```
PS C:\software-security\labs\week03-cryptography> python -c "from vulnerable_crypto import encrypt_ecb; c=encrypt_ecb(b'A'*16 + b'A'*16); print('Block 1:', c[:16].hex()); print('Block 2:', c[16:32].hex()); print('Identical:', c[:16] == c[16:32]); print('Full ciphertext:', c.hex())"
```

```
Block 1: 3bfd04cc0d7ed55358e2cbe19de21383
```

```
Block 2: 3bfd04cc0d7ed55358e2cbe19de21383
```

```
Identical: True
```

```
Full ciphertext: 3bfd04cc0d7ed55358e2cbe19de213833bfd04cc0d7ed55358e2cbe19de2138377222e061a924c591cd9c27ea163ed4
```

```
PS C:\software-security\labs\week03-cryptography> |
```

AES-ECB encrypts identical 16-byte plaintext blocks into identical ciphertext blocks when the same key is used. Because of this, repeated patterns in the original data remain visible in the encrypted data, which can reveal information about the plaintext structure (CWE-327).

Mitigation:

AES-ECB should not be used for sensitive data because it does not hide repeated patterns. An authenticated encryption mode such as AES-GCM should be used with a unique random nonce, because it provides confidentiality and also detects unauthorized modification.

Task 3 — Predictable token (15 min) · Goal: show the reset token is guessable. Steps: call `reset_token()` repeatedly; argue why a 6-digit random token (10^6 space, non-CSPRNG) is brute-forceable (CWE-330). Deliverable: sample tokens + a one-line attack estimate.

(Task 3)

```
Thura Aung | 6631503094 | 2026-08-16 15:59:54 +07:00
```

```
PS C:\software-security\labs\week03-cryptography> python -c "from vulnerable_crypto import reset_token; [print(f'Token {i+1}: {reset_token()}') for i in range(10)]"
```

```
Token 1: 576500
```

```
Token 2: 365087
```

```
Token 3: 174135
```

```
Token 4: 587473
```

```
Token 5: 235718
```

```
Token 6: 149020
```

```
Token 7: 065233
```

```
Token 8: 907873
```

```
Token 9: 733435
```

```
Token 10: 719314
```

```
PS C:\software-security\labs\week03-cryptography> |
```

The reset token contains only six decimal digits, giving just 1,000,000 possible values, and it is generated using Python's non-cryptographic random module. This makes the token unsuitable for security-sensitive authentication and potentially guessable through repeated attempts (CWE-330).

One-line attack estimate:

A 6-digit token has only 1,000,000 possibilities; at 1,000 guesses per second, all possibilities could theoretically be tested in about 16.7 minutes, with the correct token found on average in about half that time.

Mitigation:

Reset tokens should be generated using a cryptographically secure random generator such as Python's `secrets` module. The token should also have much more entropy, expire quickly, be single-use, and the application should rate-limit verification attempts.

Task 4 — Hardcoded key (5 min) · *Goal:* identify the key-management flaw. *Steps:* find `HARDCODED_KEY` in `vulnerable_crypto.py`; explain why shipping a key in source is CWE-798. *Deliverable:* the line + a 2-sentence mitigation.

(Task 4)

```
PS C:\software-security\labs\week03-cryptography> Select-String -Path .\vulnerable_crypto.py -Pattern "HARDCODED_KEY"
vulnerable_crypto.py:12:HARDCODED_KEY = b"0123456789abcdef" # MISUSE 3: hardcoded key (CWE-798)
vulnerable_crypto.py:14:     cipher = AES.new(HARDCODED_KEY, AES.MODE_ECB)

PS C:\software-security\labs\week03-cryptography> Write-Host "Thura Aung | 6631503094 | $(Get-Date -Format 'yyyy-MM-dd HH:mm:ss zzz')"
```

Storing an encryption key directly in source code is insecure because anyone who can access the source code or compiled application may recover the key. If the same key is deployed to many systems, one leaked copy can compromise all encrypted data, which is a hardcoded credential issue under CWE-798.

Mitigation:

Encryption keys should be stored outside the source code, such as in environment variables or a dedicated secret-management system. The key should also be protected with access controls and rotated if it is exposed.

Task 5 — Crack the project target's hashes (25 min) · *Goal:* apply cracking to your term project. *Steps:* **NoteVault** stores unsalted MD5 password hashes; obtain them (via the app's `/admin` once you can reach it, or from its `seed()`), and crack them with `hashcat -m 0`. *Deliverable:* the recovered password(s) + note the CWE — record this finding for your project report (`project/REPORT-TEMPLATE.md` in the repo root).

(Task 5)

```
PS D:\Downloads\Compressed\hashcat-7.1.2\hashcat-7.1.2> .\hashcat.exe -m 0 "C:\software-security\project\starter-app\project_hashes.txt" --show
15da1f78ad7d474862865bab1aab4d51:alicepw
0192023a7bbd73250516f069df18b500:admin123
PS D:\Downloads\Compressed\hashcat-7.1.2\hashcat-7.1.2> Write-Host "Thura Aung | 6631503094 | $(Get-Date -Format 'yyyy-MM-dd HH:mm:ss zzz')"
```

NoteVault stores seeded user passwords as unsalted MD5 hashes. `admin123` was recovered using RockYou, while `alicepw` required adding the known seed candidate to a small wordlist, showing that MD5 password storage is vulnerable to fast offline guessing (CWE-916/CWE-327).

Task 6 — Password storage migration (25 min) · *Goal:* fix it the way real apps do. *Steps:* write `store_password/verify_password` with `argon2id`, and a `rehash-on-login` path that upgrades a legacy

MD5 record to argon2id the next time the user logs in. *Deliverable*: the code + a short note on why migration matters.

(Task 6)

```
"""
Week 3 – FIX the misuse here. Fill in the TODOs.
pip install argon2-cffi pycryptodome
"""

import os
import hashlib
import hmac
import re

from argon2 import PasswordHasher
from argon2.exceptions import VerificationError, InvalidHashError
from Crypto.Cipher import AES

ph = PasswordHasher()

# Store all NEW passwords with Argon2id
def store_password(pw: str) -> str:
    return ph.hash(pw)

# Verify an Argon2id password
def verify_password(hash_: str, pw: str) -> bool:
    try:
        return ph.verify(hash_, pw)
    except (VerificationError, InvalidHashError):
        return False

# Detect an old MD5 hash
def is_legacy_md5(hash_: str) -> bool:
    return bool(re.fullmatch(r"[0-9a-fA-F]{32}", hash_))

# Rehash-on-login migration
def verify_and_upgrade(hash_: str, pw: str):
    # Old MD5 password
    if is_legacy_md5(hash_):
        md5_candidate = hashlib.md5(pw.encode()).hexdigest()

        if not hmac.compare_digest(md5_candidate, hash_.lower()):
            return False, hash_

    # Correct password -> replace MD5 with Argon2id
    new_hash = store_password(pw)
    return True, new_hash
```

```

# Already Argon2id
def verify_password(hash_, pw):
    if ph.check_needs_rehash(hash_):
        return True, store_password(pw)

    return True, hash_

return False, hash_

def encrypt_gcm(data: bytes, key: bytes) -> tuple[bytes, bytes, bytes]:
    nonce = os.urandom(12)
    cipher = AES.new(key, AES.MODE_GCM, nonce=nonce)
    ct, tag = cipher.encrypt_and_digest(data)
    return nonce, ct, tag

def reset_token() -> str:
    import secrets
    return secrets.token_urlsafe(16)

if __name__ == "__main__":
    # Simulate an OLD MD5 database record
    old_hash = hashlib.md5(b"password123").hexdigest()

    print("Before login:", old_hash)

    ok, upgraded_hash = verify_and_upgrade(old_hash, "password123")

    print("Login successful:", ok)
    print("After login:", upgraded_hash)
    print("Uses Argon2id:", upgraded_hash.startswith("$argon2id$"))

    # Verify the new Argon2id hash
    print("Argon2 verify:", verify_password(upgraded_hash, "password123"))

```

Rehash-on-login lets an application gradually replace weak legacy MD5 password hashes with stronger Argon2id hashes without forcing every user to reset their password immediately. When a user successfully logs in, the application verifies the old hash and then stores a new salted Argon2id hash for future logins.

Task 7 — Authenticated encryption round-trip (20 min) · *Goal:* use AEAD correctly. *Steps:* encrypt+decrypt a message with **AES-GCM** using a random 12-byte nonce and a key from an env var; then flip one ciphertext byte and show decryption **fails** (tag check). *Deliverable:* the round-trip output + the tampered-fails proof.

(Task 7)

```
PS C:\software-security\labs\week03-cryptography> python task7.py
Original message: Software Security Week 3
Nonce: dc33e549b7de4dd2288b0347
Ciphertext: 00ce65a23c1aabe1f5c0e1217213e04f48151c617594432d
Tag: 3ad1cb9aef8f1a4f552c0629ae426b7d

Decrypted message: Software Security Week 3
Round-trip: SUCCESS

Tampered ciphertext: 01ce65a23c1aabe1f5c0e1217213e04f48151c617594432d
Tamper test: SUCCESS
Decryption failed: MAC check failed
PS C:\software-security\labs\week03-cryptography> Write-Host "Thura Aung | 6631503094 | $(Get-Date -Format 'yyyy-MM-dd HH:mm:ss')"
Thura Aung | 6631503094 | 2026-08-18 01:11:02
PS C:\software-security\labs\week03-cryptography> |
```

Task 8 — TLS in practice (15 min) · Goal: read a real cert. Steps: run `openssl s_client -connect example.com:443 </dev/null 2>/dev/null | tee /tmp/tls.txt | openssl x509 -noout -issuer -subject -dates` for the cert summary, then `grep -E 'Protocol|New,' /tmp/tls.txt` for the negotiated TLS version (the version line is printed by `s_client`, not by `x509`, so the plain pipe would discard it); identify issuer, validity, and that TLS version. Deliverable: the cert summary + one line on what TLS protects that hashing/at-rest encryption does not.

(Task 8)

TLS protects data **in** transit from interception and tampering, unlike hashing or at-rest encryption, **which** do not protect data **while** it is being transmitted.

```
Acer Nitro5@LAPTOP-JJIOH995 MINGW64 /c/software-security/labs/week03-cryptograph
y (main)
$ openssl version
OpenSSL 3.2.4 11 Feb 2025 (Library: OpenSSL 3.2.4 11 Feb 2025)

Acer Nitro5@LAPTOP-JJIOH995 MINGW64 /c/software-security/labs/week03-cryptograph
y (main)
$ openssl s_client -connect example.com:443 </dev/null 2>/dev/null | tee /tmp/tl
s.txt | openssl x509 -noout -issuer -subject -dates
issuer=C=US, O=SSL Corporation, CN=Cloudflare TLS Issuing ECC CA 3
subject=CN=example.com
notBefore=Jul 29 22:10:08 2026 GMT
notAfter=Oct 27 22:17:21 2026 GMT

Acer Nitro5@LAPTOP-JJIOH995 MINGW64 /c/software-security/labs/week03-cryptograph
y (main)
$ grep -E 'Protocol|New,' /tmp/tls.txt
New, TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384

Acer Nitro5@LAPTOP-JJIOH995 MINGW64 /c/software-security/labs/week03-cryptograph
y (main)
$ echo "Thura Aung | 6631503094 | $(date '+%Y-%m-%d %H:%M:%S')"
Thura Aung | 6631503094 | 2026-08-18 14:27:29

Acer Nitro5@LAPTOP-JJIOH995 MINGW64 /c/software-security/labs/week03-cryptograph
y (main)
$
```

Task 9 — Defend / fix it (20 min) · Goal: remediate using `solution_skeleton.py`. Steps: run `python solution_skeleton.py`; confirm `store_password/verify_password` use `argon2id` (auto-salted), `encrypt_gcm` uses a random 12-byte nonce + auth tag with a key from `ENC_KEY_HEX` env, and `reset_token`

uses **secrets**. Map each fix to the CWE it closes. *Deliverable*: before/after table (misuse → fix → CWE closed)
+ screenshot of the fixed script running.

(Task 9)

Misuse	Fix	CWE closed
Weak password hashing	Argon2id with automatic salt	CWE-916
Weak/unauthenticated encryption	AES-GCM with authentication tag	CWE-327
Hard-coded encryption key	Key from ENC_KEY_HEX environment variable	CWE-798
Unsafe randomness/nonce	Random 12-byte nonce	CWE-330
Predictable reset token	secrets CSPRNG	CWE-330

```
Acer Nitro$@LAPTOP-JJIOH995 MINGW64 /c/software-security/labs/week03-cryptography
y (main)
$ pip install argon2-cffi pycryptodome
Requirement already satisfied: argon2-cffi in C:\Users\Acer Nitro5\AppData\Local\Programs\Python\Python313\Lib\site-packages (25.1.0)
Requirement already satisfied: pycryptodome in C:\Users\Acer Nitro5\AppData\Local\Programs\Python\Python313\Lib\site-packages (3.23.0)
Requirement already satisfied: argon2-cffi-bindings in C:\Users\Acer Nitro5\AppData\Local\Programs\Python\Python313\Lib\site-packages (from argon2-cffi) (25.1.0)
Requirement already satisfied: cffi>=1.0.1 in C:\Users\Acer Nitro5\AppData\Local\Programs\Python\Python313\Lib\site-packages (from argon2-cffi-bindings->argon2-cffi) (1.17.1)
Requirement already satisfied: pycparser in C:\Users\Acer Nitro5\AppData\Local\Programs\Python\Python313\Lib\site-packages (from cffi>=1.0.1->argon2-cffi-bindings->argon2-cffi) (2.22)

Acer Nitro$@LAPTOP-JJIOH995 MINGW64 /c/software-security/labs/week03-cryptography (main)
$ export ENC_KEY_HEX=$(openssl rand -hex 32)

Acer Nitro$@LAPTOP-JJIOH995 MINGW64 /c/software-security/labs/week03-cryptography (main)
$ echo $ENC_KEY_HEX
f2e75179a0bd8abada00391886c903c4cf333dd88b0f7fbbd78cb13bbd315395

Acer Nitro$@LAPTOP-JJIOH995 MINGW64 /c/software-security/labs/week03-cryptography (main)
$ python solution_skeleton.py
Password hash: $argon2id$v=19$m=65536,t=3,p=4$2niF+ael5huB0QuHYc+B9Q$NNXqw0zdGf7Daksv3EoNCfZ/efGn0V6f7yRUEwkdrvQ
Password verified: True
Nonce: 219ac7c5eda58a507add49e6
Ciphertext: 4f5b1bb0a1a0d2379dfecbd955671655527b8aa4a89076
Auth tag: 3ba2ac5391ad93869171566a7422c772
Reset token: dq-RYUT606vH0u653CmmPg

Acer Nitro$@LAPTOP-JJIOH995 MINGW64 /c/software-security/labs/week03-cryptography (main)
$ echo "Thura Aung | 6631503094 | $(date +%Y-%m-%d %H:%M:%S)"
Thura Aung | 6631503094 | 2026-08-18 14:33:13

Acer Nitro$@LAPTOP-JJIOH995 MINGW64 /c/software-security/labs/week03-cryptography (main)
$ |
```

Part 4 — Reflection

1. Map each of the four misuses to its CWE and to OWASP A04, in one line each.

- Weak password hashing → CWE-916 → OWASP A04: Cryptographic Failures
- Weak/incorrect encryption → CWE-327 → OWASP A04: Cryptographic Failures
- Predictable random values/nonces → CWE-330 → OWASP A04: Cryptographic Failures
- Hard-coded keys → CWE-798 → OWASP A04: Cryptographic Failures

2. Name a real-world breach caused by weak password hashing or hardcoded keys, and which fix here would have prevented it.

The 2012 LinkedIn breach exposed passwords stored using unsalted SHA-1 hashes, making many of them much easier to crack. Using Argon2id with automatic random salts, as **in** this lab, would have made password cracking much slower and more difficult.

3. Across all four fixes, which closes the largest real-world risk, and why?

I think replacing weak password hashing with Argon2id closes the largest real-world risk because stolen password databases can affect millions of **users**,

especially when people reuse passwords. Argon2id with salts and a deliberately expensive computation makes offline password cracking much harder.

Grading rubric (100)

Criterion	Points
Lecture questions (Part 2)	20
Exploitation + evidence (cracked hashes + ECB/token/key proof + screenshots)	40
Defense (working <code>solution_skeleton.py</code> + before/after mapping)	25
Reflection (CWE/OWASP mapping + breach + biggest-risk fix)	15

Evidence & Integrity (required)

- Identity proof:** every screenshot/diagram must show a terminal running `printf '%s | %s | ' "$$(whoami)" '<YOUR-STUDENT-ID>'; date '+%F %T %Z'` **in the same image as the evidence.** When the evidence is a browser page, a DevTools panel or a rendered response, put that terminal **beside the browser and capture the whole screen** — a cropped window carries nothing that identifies you, and the lab's own output is byte-identical for the whole cohort *by design*, so the stamp is the only thing that makes the shot yours. Generic or borrowed evidence is not accepted.
- Personalized flag (if this lab issues one):** _____ *Flags are unique per student — submitting another student's flag is a violation. How to submit: learn.zcr.ai/submit (full guide: [SUBMISSION.md](#) in the repo root).*
- Explain in your own words** (*graded on your reasoning, not copied text*):
 - What did you do, and **why did the vulnerability work?**
- I tested weak password hashing, encryption, randomness, and key handling. The vulnerabilities worked because weak hashes like MD5 are fast to crack, predictable values can be guessed, and hard-coded keys can be exposed.
 - Why does your fix actually stop it** — and what could still break it?
- I used Argon2id for passwords, AES-GCM for authenticated encryption, secrets for secure random values, and an environment variable for the encryption key. These fixes make cracking, guessing, and tampering much harder, but security could still fail if keys are leaked, nonces are reused, or the code is used incorrectly.



Audit the AI (required)

AI is a power tool you must **distrust** — you are graded on your *critique*, not the AI's answer.

- Ask an AI assistant to exploit **or** fix this week's vulnerability. Paste its full answer. I asked an AI assistant to help fix the Week 3 cryptography code. It suggested:

```
def verify_password(hash_, pw): try: return ph.verify(hash_, pw) except Exception:
return False def encrypt_gcm(data, key): nonce = os.urandom(12) cipher =
AES.new(key, AES.MODE_GCM, nonce=nonce) return nonce,
*cipher.encrypt_and_digest(data)
```

The AI said this fixed password verification and used AES-GCM securely.

2. **Find what's wrong or risky** in it — insecure code, a subtly incomplete fix, a hallucinated API/function/CVE, a missed edge case, or wrong reasoning. Quote the exact line(s).

The risky lines were:

```
"except Exception:" and "def encrypt_gcm(data, key):"
except Exception can hide other programming errors. Also, the encryption key was
passed directly to the function instead of being loaded from the required
ENC_KEY_HEX environment variable.
```

3. Produce the **correct, verified** version yourself and explain in 2–3 sentences why the AI's output was insufficient.

```
def encrypt_gcm(data): key = bytes.fromhex(os.environ["ENC_KEY_HEX"]) nonce =
os.urandom(12) cipher = AES.new(key, AES.MODE_GCM, nonce=nonce) return nonce,
*cipher.encrypt_and_digest(data)
```

I verified the fixed script in Task 9. The AI answer was incomplete because it did not enforce secure key handling and used overly broad error handling.

Disclose your AI use in the Part 1 table. This task counts toward your **Defense + Reflection** score.

Comprehension & Prompt (required)

A. Explain in Plain English (EiPE). In 2–3 sentences, in your own words, describe what this week's vulnerable code/endpoint actually *does* and *why it is exploitable* — explain the mechanism, don't dump jargon.

The vulnerable code used weak cryptography that could make passwords easier to crack or encrypted data unsafe. For example, encryption without proper authentication can allow modified ciphertext to go unnoticed.

B. Prompt Problem. Write a **single prompt** that makes an AI produce a *correct, secure* fix for one finding. Run it: does the exploit now fail? If not, refine the prompt and try again. Submit the **final prompt + the verified result**. Graded on the prompt's precision and your verification — this trains problem decomposition and AI literacy (Denny et al. 2024).

Final prompt:

Fix the encryption using AES-GCM. Use a random 12-byte nonce, load the key from ENC_KEY_HEX, return the ciphertext and authentication tag, and make decryption fail if the ciphertext is changed.

Verified result:

I ran the fixed code and the normal encrypt/decrypt round-trip succeeded. After changing one ciphertext byte, decryption failed with a MAC/tag verification error, confirming that the fix stopped the tampering attack.